

Week 2 Report Template

Planning, Setup, and MVP Development

Course: [Sum26] Practicum Project

Project name: BackTestBench: modular backtesting and sandbox platform for trading strategies

Track: Finance

TA: Gerald Imbugwa

Customer: Alexey Potemkin

Date: 15.06.2026

1. Team Information

- Team name: BackTestBench
- Primary communication channel: https://t.me/Muratich_29
- **Member 1: Marat Diarov (m.diarov@innopolis.university)**
 - Role and responsibilities: Team lead (reports, customer communication), Frontend (basic GUI)
- **Member 2: Georgii Beliaev (g.beliaev@innopolis.university)**
 - Role and responsibilities: Tech lead (defines requirements and architecture), DevOps/QA
- **Member 3: Bogdan Zinchenko (b.zinchenko@innopolis.university)**
 - Role and responsibilities: Data Engineer (manages historical data acquisition and storage).
- **Member 4: Samy Badavi (s.badavi@innopolis.university)**
 - Role and responsibilities: Simulation Engine Developer (Implements the backtesting and validation engine).
- **Member 5: Maxim Kharlamov (m.kharlamov@innopolis.university)**
 - Role and responsibilities: Analytics & Ranking Developer (Computes performance metrics and evaluates strategies).
- **Member 6: Philip Oghenerukevwe Idiare (p.idiare@innopolis.university)**
 - Role and responsibilities: Strategy Module Developer (Implements and manages trading strategies).
- **Member 7: Nikolay Taran (n.taran@innopolis.university)**
 - Role and responsibilities: Broker Adapter Developer (provides a unified interface to external market data sources and brokers and trading bots interaction with API).

2. Project and Track Statement

Project: BackTestBench: modular backtesting and sandbox platform for trading strategies

Track: Industrial

Problem statement:

Retail traders often rely on fragmented tools, broker terminals, and custom scripts to evaluate trading strategies. While existing platforms provide market access and visualization, they lack a unified environment for systematic strategy testing, comparison, and validation.

As a result, traders may struggle to assess strategy effectiveness reliably due to limited backtesting, analytics, and reproducibility. Therefore, there is a need for a modular platform that enables users to create, test, compare, and validate trading strategies on historical Russian market data and broker sandbox environments while providing transparent performance metrics.

Existing products:

Retail traders in Russia can access market data, charts, and trading functionality through brokerage platforms such as T-Investments. In addition, algorithmic traders can use broker APIs and open-source frameworks to develop and test trading strategies. However, these solutions either focus on real trading or require significant programming effort to build and evaluate strategies.

Gap/Issue/Opportunity:

There is no simple platform focused on the Russian market that allows traders to create, test, compare, and validate strategies in one place. Existing solutions provide only separate capabilities, forcing users to combine multiple tools and making strategy research more complex and less reproducible.

Non-Trivial Contribution:

BackTestBench provides a modular platform that combines:

- historical data management;
- configurable strategies;
- reproducible backtesting;
- performance analytics;
- strategy comparison and validation.

Its main contribution is an extensible architecture tailored to the Russian market with a focus on modularity, reliability, and reproducibility.

Potential Impact:

The platform can help traders evaluate strategies before deployment and serve as a foundation for future extensions such as sentiment analysis, fundamental analysis, and support for additional brokers.

3. Project Plan and Milestones

Week-by-week roadmap through the end of the course:

- **Week 1: Initialization**
Goal: decide on a project and create a team
Work: Select a project from the list, create a team, meet with the customer and write down his requirements for the final product and MVP 1, create a repository and a kanban board
Deliverables: GitHub repository, customer requirements document, kanban board, team roles distribution document
- **Week 2: Creating a product architecture**
Goal: make the architecture so that it is implementable and modular
Work: define modules and module interfaces, data flow rules, database schema, strategy configuration format, functional/non-functional requirements, static/dynamic/deployment view
Deliverables: documentation - (functional requirements, modules description, etc.) and diagrams (static/dynamic/deployment view)
- **Week 3: MVP 1 backtesting core**
Goal: deliver the first end-to-end working version of the product
Work: implement the Broker Adapter and Data Loader path for historical candles, connect one built-in strategy to the Simulation Engine, simulate trades on a historical interval, and generate a Trade Log. Keep the system focused on the core loop only.
Deliverables: a runnable MVP-1 backtest flow, one strategy executed on historical data, and a generated trade log.
- **Week 4: Validation Pipeline, Extensibility & Ranking**
Goal: transform the initial backtesting MVP into a more extensible and reliable research platform.
Work: improvement of the core architecture by extending the Broker Adapter, Data Loader, and Strategy Module. The Analytics Module will be enhanced with validation metrics and strategy ranking capabilities, while a minimal backend API and scheduler design will be prepared for future automation.
Deliverables: supported extensible strategies, validated historical data, strategy ranking, and backend access to analytics results. The foundations for automated recalculation and validation workflows will also be in place.
- **Week 5: MVP-2 Integration & User Interface**
Goal: deliver an integrated MVP-2 combining analytics, validation, automation, and visualization.
Work: The team will integrate the Strategy Module, Simulation Engine, and Trading Bot workflow into a unified validation process. The Analytics Module will be finalized

for reporting, while the Scheduler and Notifications modules will provide automated execution and monitoring. A simple frontend will be connected to the backend to visualize strategy performance and rankings.

Deliverables: MVP-2 workflow with strategy validation, automated recalculation, analytics reporting, and a user interface for viewing results and rankings.

- **Week 6: Stabilization, QA, and release preparation**

Goal: polish the product and prepare a credible final release.

Work: fix integration issues, improve validation and error handling, verify reproducibility, run end-to-end testing, and finalize documentation. Prepare the final demo and explain what is already implemented versus what remains as future work.

Deliverables: stable final build, deployment/README documentation, test evidence, and demo-ready release.

Major milestones:

- 1) **Architecture freeze:** after Week 2, the team should have a stable design and interface contracts
- 2) **First end-to-end backtest:** by Week 3, the product should already run one strategy on historical data and produce a Trade Log. This is the main technical milestone.
- 3) **Metrics and persistence layer:** by Week 4, the product must be modular and reliable
- 4) **Usable trader-facing UI:** by Week 5, the product must meet all customer requirements, both functional and non-functional
- 5) **Final release candidate:** by Week 6, the system should be stable enough for demonstration and handover

Dependencies:

- Simulation Engine depends on the Data Loader and Broker Adapter, because it needs clean historical candles.
- Analytics depend on Trade Log generation, because metrics are computed from executed trades.
- Frontend depends on persisted results and stable API/schema, because it reads strategy runs, metrics, and rankings.
- Any future ranking / scheduler / sandbox validation depends on the core backtest loop, so those should be treated as extensions on top of a working MVP-1 core.

Known risks and mitigation:

- External data/API instability: the broker or market data source may be unavailable or inconsistent, so the Data Loader and error handling must be resilient.
- Data quality issues: gaps, duplicates, and invalid values in historical data can distort backtest results, which is why validation is critical.
- Scope creep: the product includes advanced ideas such as scheduler-based recalculation, sandbox validation, notifications, and future analytics modules, but these can easily exceed course time if not controlled.
- Reproducibility risk: if strategy versioning, parameters, or input data are not saved carefully, results may not be reproducible.

- Integration complexity: even with a modular design, connecting the modules cleanly is often harder than implementing each module separately.
- Overpromising automation: the scheduler/top-N/trading-bot loop is a strong extension, but it should only be committed to if the core pipeline is already stable.

Kanban board link: <https://github.com/orgs/BackTest-bench-team/projects/1>

4. Requirements Definition

User stories:

- 1) As a user, I want to see how the best strategy is performing right now, so that I can quickly understand whether it is worth considering. (MVP 2 scope)
Acceptance criteria:
 - The system shows the currently best strategy
 - The system displays basic performance metrics, at minimum P&L.
 - The result is based on real time trading data provided by trading bot working with T-Bank API or any other.
- 2) As a strategy creator, I want to create my own strategy and test it, so that I can evaluate my ideas on historical market data. (MVP 2 scope)
Acceptance criteria:
 - The user can define or select a strategy configuration.
 - The system runs a backtest on a selected historical interval.
 - The system produces a trade log and stores the result.
- 3) As a user, I want the system to show only strategies that can beat a bank deposit return, so that I can focus on strategies with acceptable performance. (MVP 2 scope)
Acceptance criteria:
 - The system calculates strategy performance against a deposit benchmark.
 - Strategies below the benchmark are excluded from the top ranking or clearly marked.
 - The ranking is recalculated automatically when new results are available.

MVP scope:

MVP 1: one working strategy, a working backtest engine, trade log generation, minimal metrics calculation from the TradeLog, broker integration, and a simple frontend for analytics and strategy description.

Out of MVP 1: multiple strategies, full strategy comparison, full Top-N ranking logic, scheduler-based recalculation, trading bot sandbox validation, notifications, advanced analytics modules, and broader broker support beyond the first integration. These items exist in the broader product vision, but they belong to later iterations.

5. Project Setup

Repository / codebase structure:

- GitHub organisation: <https://github.com/BackTest-bench-team> / Repository: <https://github.com/BackTest-bench-team/BackTestBench>. The repo is structured as a Python monorepo: /src contains one sub-package per module (broker_adapter, data_loader, strategy, engine, analytics); /tests mirrors /src for unit and integration tests; interfaces.py at the root defines all shared dataclasses and ABCs; docker-compose.yml at the root brings up the full stack.

Runnable boilerplate or hello-world proof:

- Not applicable for Week 2. No runnable code exists. The repository contains only empty placeholder files and documentation.

Docker support / run instructions:

- Not applicable for Week 2. docker-compose.yml is committed as an empty placeholder. The full Docker setup will be completed in Week 3.
- Branch strategy: main (stable, protected) and feat/* branches. Direct push to main is blocked; all changes go through a PR and require at least one approval.

6. Tech Stack Selection

- Frontend: Next.js (React, TypeScript). Grafana as a fallback if the UI is not ready in time for a demo.
- Backend: Python 3.11 with FastAPI. Natural fit for the data-heavy workload; auto-generates OpenAPI docs at /docs.
- Database: SQLite for MVP-1 (zero setup, single file). PostgreSQL for MVP-2 when concurrent access or data volume requires it. SQLAlchemy ORM throughout so the switch is a one-line config change.
- Infrastructure / deployment: Docker Compose on the team shared server. One compose file covers all containers (api, db, scheduler, frontend).
- Other tools / libraries: pandas and numpy for vectorised candle processing; pytest for unit and integration tests; GitHub Actions for CI (lint + tests on every PR); APScheduler for the recalculation job.
- Short justification: Python is the standard language for quant/data work and is the most familiar to the team. FastAPI is lightweight and self-documenting. SQLite removes infrastructure overhead for MVP-1. Docker Compose gives everyone a reproducible environment in one command.

7. Initial Design and Skeleton

- Frontend structure and routing: Next.js app router with three main pages: / (dashboard showing top-N strategies and latest metrics), /backtest (form to select strategy, instrument, and date range, and launch a run), /results/[runId] (detailed view of a single backtest run: trade log table, P&L chart, metrics summary). All pages read data from the FastAPI backend over HTTP.
- Backend API contract: REST API defined in FastAPI with automatic OpenAPI docs at /docs. Main endpoints: POST /backtest/run (start a backtest run, returns run_id), GET /backtest/{run_id} (poll status and retrieve TradeLog + MetricsReport), GET /strategies (list registered strategies), GET /top-n (return current top-N ranked strategies). Full schema is in the repo at /docs/api.md.

- Initial database schema: five tables in SQLite (MVP-1): candles (instrument, timeframe, timestamp, open, high, low, close, volume), strategies (id, name, yaml_config, created_at), backtest_runs (id, strategy_id, instrument, from_dt, to_dt, status), trades (id, run_id, entry_price, exit_price, quantity, pnl, opened_at, closed_at), metrics (id, run_id, total_pnl, sharpe_ratio, max_drawdown, win_rate, deposit_baseline_pnl). The top_n and alert_log tables are added in MVP-2.
- Design artifacts / wireframes / screenshots: Architecture diagrams (static view, dynamic view, deployment view) are committed to docs/Product description.docx in the repository.

8. Core MVP Implementation

- Implemented features: No code has been implemented in Week 2. The week focused entirely on architecture, contracts, and project setup. The following artifacts are committed to the repository: (1) interfaces.py — empty file, placeholder for the shared dataclass and ABC definitions to be written in Week 3. (2) docs/api.md — full REST API contract covering all planned endpoints with request/response schemas. (3) docs/database_schema.md — SQLite schema for MVP-1 covering candles, strategies, backtest_runs, trades, and metrics tables. (4) Full repository folder structure (src/, tests/, config/, docs/, scripts/, .github/workflows/) with .gitkeep files so the layout is visible in the repo. (5) docker-compose.yml — empty file, to be filled in Week 3.
- Functional end-to-end user journey: (for week 3) user will see set of strategies and their performance (top N strategies)
- Error handling and auth considerations: Not applicable for Week 2. Error handling will be implemented alongside each module starting in Week 3. No authentication is planned for the course prototype. The API error format (structured JSON with error and detail fields) is defined in docs/api.md.
- Frontend-backend integration status: Not applicable for Week 2. No frontend or backend code exists yet. Integration is planned for Week 5.
- Data storage / retrieval status: The database schema is documented in docs/database_schema.md. No SQLAlchemy models or actual database have been created yet. The schema covers five tables for MVP-1: candles, strategies, backtest_runs, trades, and metrics. Implementation starts in Week 3.

9. Track-Specific Core Work — Industrial

- Integrated contribution into or against the existing product: The project is greenfield. The external integration point is the T-Bank Investments sandbox API, used through the BrokerAdapter abstraction. In Week 2 the API contract and adapter interface were defined in docs/api.md and interfaces.py.

- Initial measurable comparison to the baseline: Not available in Week 2. The first measurable comparison (strategy P&L vs. 13% annual deposit baseline) will be produced in Week 3 once the full backtest pipeline runs end-to-end.
- Baseline used for comparison: A 13% annual deposit return (the approximate Russian bank deposit rate referenced by the customer). Strategies whose total_pnl is below deposit_baseline_pnl in the MetricsReport are excluded from the top-N ranking.

10. Internal Demo and Feedback

- Demo date: No internal demo was held in Week 2. A customer meeting was conducted on 14.06.2026 where the architecture, module interfaces, and project plan were reviewed and approved.
- Main issues found: Customer noted that the GitHub board statuses were not up to date and asked the team to keep them current. No blocking technical issues were found during the architecture review.
- Immediate improvements: Update GitHub Issues statuses to reflect actual progress. Add missing issues for Week 2 deliverables (interfaces.py, DB schema, API contract, folder structure). Begin Week 3 implementation tasks.
- Next steps: Implement interfaces.py, connect Data Loader to Simulation Engine, run MA Crossover on real historical data, produce a TradeLog and MetricsReport, display basic metrics via the API, and conduct the first customer demo.

11. Required Links

- Kanban board link: <https://github.com/orgs/BackTest-bench-team/projects/1>
- Relevant PRs / issues: <https://github.com/BackTest-bench-team/BackTestBench/issues>
- API documentation: docs/api.md in the repository:
<https://github.com/BackTest-bench-team/BackTestBench/blob/main/docs/api.md>
- Screenshots / GIFs / demo artifacts: No screenshots available in Week 2. Architecture diagrams (static, dynamic, deployment views) are in docs/Product description.docx in the repository.